The Journal of Open Source Software

CMBLensing.py: A Python Package for MAP Estimation of CMB Fields and Cosmological Parameter Sampling

Zane Blood [1¶], **Kimmy Wu** [1], **and Fei Ge** [1]

1 California Institute of Technology ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Weak gravitational lensing by large-scale structure distorts the cosmic microwave background (CMB), and isolating the unlensed CMB and the lensing potential ϕ that deflects it is an important part of extracting cosmological information from CMB maps. CMBLensing.py is a Python port of CMBLensing.jl (Marius Millea, 2017), a Julia package that simulates lensed CMB maps, computes joint Maximum A Posteriori (MAP) estimates of the unlensed CMB and ϕ, and samples cosmological parameters. Both codebases operate in the flat-sky approximation, which assumes the patches of the sky being analyzed are small enough that curvature can be neglected. CMBLensing.py is built on JAX (Bradbury et al., 2018), so it is just-in-time (JIT) compiled, differentiable, and runs on both CPU and GPU. It reproduces the core functionality of the Julia package and has been thoroughly tested for regressions. We have also extended the sampler, which originally inferred the tensor-to-scalar ratio r and the lensing amplitude A_ϕ (Marius Millea et al., 2020), to jointly sample any subset of five of the ΛCDM parameters $\{100 * \theta_{MC}, \ln(A_s * 10^{10}), n_s, \Omega_b h^2, \Omega_c h^2\}$, with the optical depth to reionization τ held fixed.

Statement of Need

CMB analyses increasingly combine physical forward models with machine-learned components, differentiable emulators, and gradient-based samplers, and much of this tooling is now written in JAX (Chanial et al., 2026; Zhou et al., 2026). A Julia implementation cannot take part in such pipelines: it cannot be traced by `jax.jit` or differentiated by `jax.grad`, and calling it from Python through `juliacall` (Rowley, 2021) adds Julia compilation latency at the start of every session and type conversions at every call. CMBLensing.py removes these barriers by providing the same lensing forward model, MAP estimator, and sampler natively in JAX. It is intended for researchers performing field-level CMB lensing reconstruction, delensing, or cosmological inference.

State of the Field

CMBLensing.jl has been publicly available since 2017 and has been used in a number of research applications such as (M. Millea et al., 2021), which inferred A_ϕ from SPTpol polarization data. Other public tools address similar problems: plancklens (Planck Collaboration et al., 2020) provides quadratic-estimator lensing reconstruction, LensIt (Carron & Lewis, 2017) and delensalot (Belkner et al., 2024) perform iterative MAP lensing reconstruction, lenspyx (Reinecke et al., 2023) offers curved-sky lensing methods, and Flinch (Crespi et al., 2025) performs field-level parameter inference. CMBLensing.py differs from these packages in a few

DRAFT

The Journal of Open Source Software

different ways. First, its forward model and posterior are automatically differentiable and run natively on GPU, whereas plancklens, Lenslt, lenspyx, and delensalot are not automatically differentiable. Second, because it is written in JAX, it composes directly with other JAX projects: CMBLensing.jl and Flinch are differentiable but are written in Julia, which makes interfacing with Python packages difficult. Third, it jointly samples ΛCDM parameters together with the unlensed CMB field f and lensing potential ϕ, whereas the CMBLensing.jl sampler pairs f and ϕ only with r and A_ϕ. Fourth, unlike Flinch, it includes lensing in the forward model and samples ϕ.

Software Design

CMBLensing.py is organized around flat-sky field and operator types. Maps are `Field` objects (`FlatS0` for temperature, `FlatS2` for polarization, `FlatS02` for both), and covariances, beams, and masks are Fourier-space operators (`DiagonalScalar`, `DiagonalEB`, `BlockTEB`). We use `flax.struct` dataclasses (Heek et al., 2026), so they can easily pass through `jax.jit`, `jax.grad`, and `jax.vmap`. Arithmetic is overloaded per type, so the same lensing, Wiener-filter, and likelihood code serves all three polarization configurations without branching.

Each module was ported and validated against CMBLensing.jl on identical inputs. A pytest (Krekel & others, 2004) suite compares the Python outputs to cached Julia results and checks that they agree to numerical precision. In addition to comparing the accuracy of the two implementations, we also performed a runtime comparison of the MAP estimation method `map_joint`. We find that on 4 CPU cores, `map_joint` applied to temperature-plus-polarization data runs 1.4–2x slower than the Julia version for maps at or below 512×512 pixels and 3.8x slower for 1024×1024 pixel maps. The times quoted are for the compiled versions of both code bases, timed after an initial uncompiled run. This slowdown is mainly because of the line search step for optimizing ϕ in the `map_joint` algorithm. Brent's method in CMBLensing.jl converges in markedly fewer evaluations than the standard JAX and SciPy line searches available to us.

The Python port's parameter sampler keeps the same Gibbs structure of (Marius Millea et al., 2020) and replaces its (r, A_ϕ) step with a Metropolis-within-Gibbs update of each sampled ΛCDM parameter. The pseudocode below describes how the algorithm draws parameter samples for an input data map d:

Initialize the starting guesses for the lensing potential ϕ and the ΛCDM parameters $\vec{\theta}$
for $k = 1, \ldots, N_{\text{steps}}$ **do**
  Sample $f \sim P(f \mid d, \phi, \vec{\theta})$ via conjugate gradient
  Sample $\phi \sim P(\phi \mid d, f, \vec{\theta})$ via Hamiltonian Monte Carlo
  for each $\theta_i \in \vec{\theta}$ **do**
    Sample $\theta_i \sim P(\theta_i \mid d, f, \phi, \{\theta_j\}_{j \neq i})$ via 1-D Metropolis-Hastings
  end for
end for

Because every θ proposal needs new unlensed spectra, and calling CAMB at each step would dominate the runtime, we precompute CAMB spectra on a five-dimensional grid over $\{H_0, \ln(A_s * 10^{10}), n_s, \Omega_b h^2, \Omega_c h^2\}$ and evaluate a tensor-product cubic spline through it. The 8 GB grid is hosted on Hugging Face and downloaded on first use.

To judge whether the sampler performs optimally, a Fisher forecasting script in the repository computes finite differences of the delensed temperature and ϕ covariance matrices following (Taylor et al., 1997). The forecast is very sensitive to how delensing and the lensing reconstruction noise are modeled. We find that the delensed temperature covariance matrix must be computed with our own implementation of the LenseFlow algorithm (Marius Millea et al., 2019) rather than with CAMB's delensing software (Lewis et al., 2000; Lewis, 2026). We also model the ϕ reconstruction noise with the quadratic-estimator noise bias N_0 (Hu & Okamoto, 2002), with both the filter and the response set to the unlensed temperature

DRAFT

covariance matrix. This choice gives the tightest of the forecasts the script supports, and it is the one shown in Figure 1.

Research Impact Statement

The ΛCDM extension of the Gibbs sampler makes CMBLensing.py, to our knowledge, the only public code that draws joint posterior samples of the ΛCDM parameters, the unlensed CMB, and the lensing potential, with CMB lensing in the forward model. Our group has used the code to jointly infer $\{100 * \theta_{\mathrm{MC}},\ \ln(A_s * 10^{10}),\ \Omega_c h^2\}$ from synthetic lensed temperature maps of 128×128 pixels at 2.5 arcmin/pixel resolution, with white noise levels of 2.5, 5, and 10 μK-arcmin, and the remaining ΛCDM parameters fixed at their ground-truth values. We present the results here as a worked example of the pipeline.

For each noise level we ran five sub-chains on each of ten independent map realizations, starting from random points in the prior range. The full configuration and tuned sampler settings are listed in the `experimental_results/` folder of the repository. After removing burn-in, each map's sub-chains were thinned by their integrated autocorrelation time and concatenated into one chain per map, and a Gaussian kernel density estimate of each chain gave one posterior per map. The product of these ten posteriors then represents the posterior given all ten maps, and Table 1 summarizes the results for each noise level tested. All post-processing is automated by a script in the repository.

Table 1: Learned posteriors given all ten maps at each noise level. Ground truths are $100 * \theta_{\mathrm{MC}} = 1.031732$, $\ln(A_s * 10^{10}) = 3.218387$, and $\Omega_c h^2 = 0.109381$. Both the mean and standard deviation listed are for the product of the ten per-map posteriors. The Z-score is the absolute value of the offset of the mean from the ground truth in units of the standard deviation listed. Numbers in parentheses are bootstrapped Monte Carlo errors on the last digit, e.g. $1.0318(1) = 1.0318 \pm 0.0001$.

Parameter	Statistic	2.5 μK-arcmin	5 μK-arcmin	10 μK-arcmin
$100 * \theta_{\mathrm{MC}}$	Mean	$1.0318(1)$	$1.0314(2)$	$1.0305(2)$
	Standard deviation	$0.00065(5)$	$0.00104(8)$	$0.00141(8)$
	Z-score	$0.1(2)$	$0.3(2)$	$0.8(1)$
$\ln(A_s * 10^{10})$	Mean	$3.223(1)$	$3.224(1)$	$3.226(1)$
	Standard deviation	$0.0055(6)$	$0.0062(4)$	$0.0072(4)$
	Z-score	$0.8(3)$	$0.8(2)$	$1.0(2)$
$\Omega_c h^2$	Mean	$0.1092(1)$	$0.1089(1)$	$0.1082(2)$
	Standard deviation	$0.00067(5)$	$0.00087(5)$	$0.00147(9)$
	Z-score	$0.2(2)$	$0.6(1)$	$0.8(1)$

All parameters are recovered within 1σ of the ten-map posterior, i.e. within about 0.3σ of a single map's posterior. As one would expect, the constraints on the learned parameters become tighter as the noise level is lowered.

For each configuration, the Fisher forecast described in the Software Design section predicts the width of a single map's posterior. Figure 1 compares it with the chains at 5 μK-arcmin: the empirical single-map widths agree with the forecast to within 10% for this specific trial, indicating that the sampler is operating close to optimally.

DRAFT

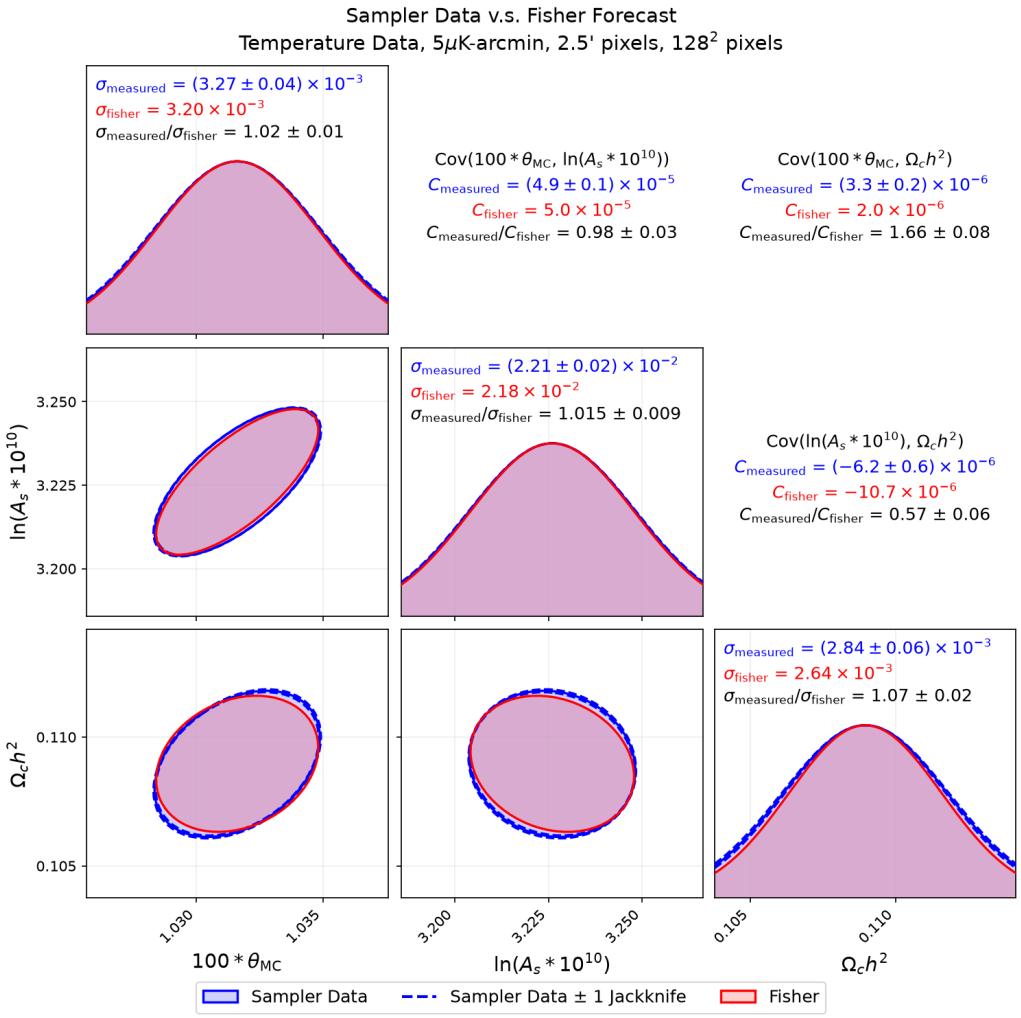


Figure 1: Single-map posteriors from the chains (blue) and the Fisher forecast (red) at 5 μK-arcmin, showing 68% confidence contours. This forecast used the delensed temperature covariance via `lense_flow` as the source of temperature information and the white noise covariance as the source of noise in the temperature field. Mismodeling the delensing in our sampler by using either the fully lensed or unlensed temperature covariance in the temperature information block of the forecast can lead to errors in the forecast degeneracies and widths - possibly flipping the sign of parameter covariances. We use the ϕ covariance as the source of lensing potential information and the quadratic estimate noise bias N_0 as the source of noise in the lensing potential. We set both the response and filter in N_0 to the unlensed temperature covariance which represents a lower bound on the noise in ϕ.

Limitations

Without significantly redesigning the sampling algorithm in CMBLensing.py, we have been able to sample at most 3 LCDM parameters concurrently without seeing significant degeneracies and/or biases between parameters. Breaking these degeneracies typically requires moving to larger maps. One iteration of the sampler running on 4 CPU cores takes under 3 seconds for 128×128 pixel temperature-only maps on Caltech's Resnick HPC (Caltech Resnick High Performance Computing Center, 2026) or Texas A&M's ACES cluster (Boerner et al., 2023), but the runtime grows quickly with map size. The main bottleneck is the HMC step for the lensing potential. Other works such as (Crespi et al., 2025) report large efficiency gains from MicroCanonical Langevin Monte Carlo, and alternative samplers for ϕ are a natural direction

for future work.

We find that at low noise levels, the auto-correlation length of the parameter samples is very long, thus making the effective number of independent samples small and the estimate of the posterior noisy. One reason for this high correlation length is the current mixing matrix design. We adopted the mixing matrices D and G from (Marius Millea et al., 2020) which are built from the noise covariances. As such, they work well at high noise where the parameter constraints are weak, but they lose effectiveness as the noise drops. A path to improve the autocorrelation length is a mixing scheme that is independent of the noise level. We leave that for future work.

AI Usage Disclosure

Claude Code (Anthropic) was used to generate and refactor code throughout the codebase, to write tests and documentation, and to help prepare this manuscript. All AI-generated code was reviewed by the authors before being merged, and the authors take full responsibility for the software and the manuscript.

Acknowledgements

The computations presented here were conducted in part at the Resnick High Performance Computing Center, a facility supported by the Resnick Sustainability Institute at the California Institute of Technology.

This work also used the ACES cluster at Texas A&M University through allocation PHY260360 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program (Boerner et al., 2023), which is supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296.

The authors acknowledge support from an Early Career Research Award DE-SC0026376 of the Department of Energy

References

Belkner, S., Carron, J., Legrand, L., Umiltà, C., Pryke, C., & Bischoff, C. (2024). CMB-S4: Iterative internal delensing and r constraints. *The Astrophysical Journal*, *964*, 148. https://doi.org/10.3847/1538-4357/ad2351

Boerner, T. J., Deems, S., Furlani, T. R., Knuth, S. L., & Towns, J. (2023, July). ACCESS: Advancing innovation: NSF's advanced cyberinfrastructure coordination ecosystem: Services & support. *Practice and Experience in Advanced Research Computing (PEARC '23)*. https://doi.org/10.1145/3569951.3597559

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Katariya, Y., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/jax-ml/jax

Caltech Resnick High Performance Computing Center. (2026). *Caltech resnick high performance computing center*. https://www.hpc.caltech.edu

Carron, J., & Lewis, A. (2017). Maximum a posteriori CMB lensing reconstruction. *Physical Review D*, *96*(6), 063510. https://doi.org/10.1103/PhysRevD.96.063510

Chanial, P., Biquard, S., Kabalan, W., Sohn, W., Basyrov, A., Beringue, B., Boucaud, A., Landais, A., Morshed, M., Stompor, R., Tsang King Sang, E., Villarrubia-Aguilar, A., &

DRAFT

Errard, J. (2026). *Furax: A modular JAX framework for linear operators in astrophysical and cosmological data analysis*. https://doi.org/10.48550/arXiv.2603.19600

Crespi, A., Bonici, M., Loureiro, A., Ruiz-Zapatero, J., Sladoljev, I., Li, Z., Bayer, A., Millea, M., & Seljak, U. (2025). *Flinch: A differentiable framework for field-level inference of cosmological parameters from curved sky data*. https://doi.org/10.48550/arXiv.2510.26691

Heek, J., Levskaya, A., Oliver, A., Ritter, M., Rondepierre, B., Steiner, A., & Zee, M. van. (2026). *Flax: A neural network library and ecosystem for JAX* (Version 0.12.12). http://github.com/google/flax

Hu, W., & Okamoto, T. (2002). Mass reconstruction with Cosmic Microwave Background polarization. *The Astrophysical Journal*, *574*(2), 566–574. https://doi.org/10.1086/341110

Krekel, H., & others. (2004). Pytest. In *GitHub repository*. GitHub. https://github.com/pytest-dev/pytest

Lewis, A. (2026). CAMB v2: Cosmological power spectra for high-precision surveys. *Journal of Cosmology and Astroparticle Physics*, *2026*(10), 013. https://doi.org/10.1088/1475-7516/2026/10/013

Lewis, A., Challinor, A., & Lasenby, A. (2000). Efficient computation of CMB anisotropies in closed FRW models. *The Astrophysical Journal*, *538*(2), 473–476. https://doi.org/10.1086/309179

Millea, Marius. (2017). CMBLensing.jl. In *GitHub repository*. GitHub. https://github.com/marius311/CMBLensing.jl

Millea, Marius, Anderes, E., & Wandelt, B. D. (2019). Bayesian delensing of CMB temperature and polarization. *Physical Review D*, *100*(2), 023509. https://doi.org/10.1103/PhysRevD.100.023509

Millea, Marius, Anderes, E., & Wandelt, B. D. (2020). Sampling-based inference of the primordial CMB and gravitational lensing. *Physical Review D*, *102*(12), 123542. https://doi.org/10.1103/PhysRevD.102.123542

Millea, M., Daley, C. M., Chou, T.-L., Anderes, E., Ade, P. A. R., Anderson, A. J., Austermann, J. E., Avva, J. S., Beall, J. A., Bender, A. N., Benson, B. A., Bianchini, F., Bleem, L. E., Carlstrom, J. E., Chang, C. L., Chaubal, P., Chiang, H. C., Citron, R., Corbett Moran, C., … Yefremenko, V. (2021). Optimal Cosmic Microwave Background lensing reconstruction and parameter estimation with SPTpol data. *The Astrophysical Journal*, *922*(2), 259. https://doi.org/10.3847/1538-4357/ac02bb

Planck Collaboration, Aghanim, N., Akrami, Y., Ashdown, M., Aumont, J., Baccigalupi, C., & others. (2020). Planck 2018 results. VIII. Gravitational lensing. *Astronomy & Astrophysics*, *641*, A8. https://doi.org/10.1051/0004-6361/201833886

Reinecke, M., Belkner, S., & Carron, J. (2023). Improved cosmic microwave background (de-)lensing using general spherical harmonic transforms. *Astronomy & Astrophysics*, *678*, A165. https://doi.org/10.1051/0004-6361/202346717

Rowley, C. (2021). PythonCall.jl. In *GitHub repository*. GitHub. https://github.com/JuliaPy/PythonCall.jl

Taylor, A., Heavens, A., Ballinger, B., & Tegmark, M. (1997, April). Cosmological parameter estimation from the CMB. *Proceedings of the Particle Physics and Early Universe Conference (PPEUC)*. https://doi.org/10.48550/arXiv.astro-ph/9707265

Zhou, Z., Giovanetti, C., & Liu, H. (2026). ABCMB: A Python+JAX package for the Cosmic Microwave Background power spectrum. *Journal of Cosmology and Astroparticle Physics*, *2026*(8), 078. https://doi.org/10.1088/1475-7516/2026/08/078

DRAFT